Bài 2.16 – Feature Engineering (Kỹ thuật đặc trưng)

> **Giai đoạn 2 – Machine Learning cổ điển** · Thời lượng: 2 buổi · Hình thức mở đầu: *Câu chuyện*

🎯 Sau bài này bạn sẽ

- Hiểu vì sao **đặc trưng tốt thường quan trọng hơn thuật toán tốt**.
- Nắm "hộp đồ nghề": tỉ lệ, log, chia nhóm, ngày giờ (kể cả mã hóa vòng tròn sin/cos), tương tác, mã hóa cột nhiều nhóm.
- Biết đặc trưng nào hợp với **mô hình tuyến tính**, đặc trưng nào hợp với **mô hình cây**.
- **Kiểm chứng** mọi đặc trưng mới bằng số liệu, và chấp nhận khi nó không giúp gì.

Chia buổi:

- **Buổi 1:** Mục 1 → 5 (tư duy, tỉ lệ, log, chia nhóm, ngày giờ).
- **Buổi 2:** Mục 6 → 10 (cột nhiều nhóm, đặc trưng nghiệp vụ, kiểm chứng, bài tập).

---

1. Câu chuyện: Cuộc thi dự đoán giá taxi

Một công ty taxi tổ chức cuộc thi: dự đoán **giá cuốc xe** từ dữ liệu: tọa độ đón, tọa độ trả, thời điểm đón, số khách.

Đội A gồm các tiến sĩ, dùng mô hình mạng nơ-ron phức tạp nhất, đưa thẳng 6 cột thô vào. Kết quả: trung bình.

Đội B gồm một cựu tài xế taxi và một sinh viên. Họ dùng mô hình đơn giản, nhưng **tạo thêm đặc trưng** từ kinh nghiệm chạy xe:

- **Khoảng cách** giữa điểm đón và điểm trả (giá taxi tính theo km!).
- **Giờ cao điểm** (kẹt xe → tính thêm tiền chờ).
- **Đi sân bay hay không** (giá cố định).
- **Cuối tuần, đêm khuya** (phụ phí).

Đội B thắng xa. Mô hình của đội A phải **tự mò** ra công thức "khoảng cách = $\sqrt{(\Delta\text{kinh độ}^2 + \Delta\text{vĩ độ}^2)}$" từ hàng triệu ví dụ, trong khi đội B **đưa thẳng** đáp án đó cho mô hình.

> **"Mô hình chỉ giỏi bằng đặc trưng bạn đưa cho nó."** Feature Engineering là nơi **hiểu biết nghiệp vụ** của con người được chuyển thành sức mạnh cho máy.

---

2. Khái niệm

- **Feature Engineering (Kỹ thuật đặc trưng):** Dùng hiểu biết về dữ liệu và nghiệp vụ để **tạo, biến đổi, chọn lọc** đặc trưng, giúp mô hình học dễ hơn.
- **Ví dụ đời thực:** Bác sĩ không chỉ nhìn cân nặng và chiều cao riêng lẻ, mà tính **BMI = cân nặng / chiều cao²**. Một con số mới, nói lên nhiều hơn hai con số gốc cộng lại.

Bạn đã làm Feature Engineering từ Giai đoạn 1:

Đặc trưng	Bài	Loại
`phong_moi_ho = tong_phong / so_ho`	1.4	Tỉ lệ
`km_tp_lon` (khoảng cách tới LA/SF)	1.12	Nghiệp vụ / địa lý
One-hot, ordinal	1.6	Mã hóa cột chữ
Đa thức x^2, $x \cdot y$	2.6	Tương tác
`o_do_thi` từ DBSCAN	2.14	Từ mô hình không giám sát

---

3. Đặc trưng tỉ lệ và log

3.1. Chuẩn bị dữ liệu "thô"

Bài này bắt đầu lại từ dữ liệu gốc (chỉ bỏ các khu bị giới hạn trần giá), để thấy rõ tác dụng của từng đặc trưng.

```
import numpy as np
import pandas as pd

url = "https://download.mlcc.google.com/mledu-
datasets/california_housing_train.csv"
raw = pd.read_csv(url)
raw.columns = ["kinh_do", "vi_do", "tuoi_nha", "tong_phong", "tong_phong_ngu",
               "dan_so", "so_ho", "thu_nhap", "gia_nha"]
raw = raw[raw["gia_nha"] < 500001].reset_index(drop=True)
raw["gia_nha"] = raw["gia_nha"] / 1000

def them_dac_trung(d):
    """Tạo các đặc trưng mới. Viết thành hàm để áp dụng giống hệt cho dữ liệu mới
sau này."""
    d = d.copy()
    d["phong_moi_ho"] = d["tong_phong"] / d["so_ho"]            # tỉ lệ: nhà rộng
hay chật
    d["ti_le_phong_ngu"] = d["tong_phong_ngu"] / d["tong_phong"]
    d["nguoi_moi_ho"] = d["dan_so"] / d["so_ho"]
    km = lambda v1, k1, v2, k2: np.sqrt(((k1 - k2) * 111 * np.cos(np.radians(v1)))
** 2 + ((v1 - v2) * 111) ** 2)
    d["km_LA"] = km(d["vi_do"], d["kinh_do"], 34.05, -118.24)
    d["km_SF"] = km(d["vi_do"], d["kinh_do"], 37.77, -122.42)
    d["km_tp_lon"] = d[["km_LA", "km_SF"]].min(axis=1)
    for c in ["phong_moi_ho", "nguoi_moi_ho", "km_tp_lon", "thu_nhap"]:
        d["log_" + c] = np.log1p(d[c])                                # log: nén giá
trị lớn (bài 1.7)
    return d

df = them_dac_trung(raw)
y = df["gia_nha"]
```

3.2. Log giúp lộ ra quan hệ

```
print(df[["phong_moi_ho", "log_phong_moi_ho"]].corrwith(y).round(3).to_dict())
print(df[["nguoi_moi_ho", "log_nguoi_moi_ho"]].corrwith(y).round(3).to_dict())
```

Kết quả mẫu:

```
{'phong_moi_ho': 0.108, 'log_phong_moi_ho': 0.177}
{'nguoi_moi_ho': -0.037, 'log_nguoi_moi_ho': -0.238}
```

`nguoi_moi_ho` gần như không tương quan với giá (−0,037), nhưng **log** của nó tương quan −0,238, gấp hơn 6 lần. Vài khu cực đoan (ký túc xá 502 người/hộ, bài 1.12) đã che mất quan hệ thật. Log "nén" chúng lại, quan hệ lộ ra. (Giống như Spearman ở bài 1.7 phát hiện.)

---

4. Chia nhóm (Binning)

- **Định nghĩa:** Biến một cột số liên tục thành các **khoảng** (nhóm).
- **Khi nào dùng:** Quan hệ "bậc thang" hoặc "chữ U" mà mô hình tuyến tính không học được; hoặc để giải thích dễ hơn ("khách 25–34 tuổi").

```
df["nhom_tuoi_nha"] = pd.cut(df["tuoi_nha"], bins=[0, 10, 20, 30, 40, 52],
                             labels=["0-10", "11-20", "21-30", "31-40", "41-52"])
print(df["nhom_tuoi_nha"].value_counts().sort_index().to_dict())
```

Kết quả mẫu:

```
{'0-10': 1273, '11-20': 3806, '21-30': 3800, '31-40': 4396, '41-52': 2911}
```

Sau đó one-hot cột nhóm này cho mô hình tuyến tính. Mỗi nhóm có một hệ số riêng, nên mô hình học được quan hệ **bất kỳ hình dạng**, không bị ép thành đường thẳng.

`pd.cut`	`pd.qcut`
Chia theo **ngưỡng** bạn chọn	Chia theo **phân vị**: mỗi nhóm có số lượng gần bằng nhau
Ví dụ: tuổi 18–25, 26–35...	Ví dụ: 4 nhóm thu nhập, mỗi nhóm 25% khách

> ⚠️ Mô hình cây **tự chia nhóm** rất giỏi (bài 2.9). Binning chủ yếu giúp mô hình tuyến tính, hoặc giúp giải thích.

---

5. Đặc trưng ngày giờ

Cột thời gian **không thể** đưa thẳng vào mô hình. Phải tách thành các thông tin có ý nghĩa.

```
don = pd.DataFrame({"thoi_gian": pd.to_datetime(["2026-11-09 07:30", "2026-11-14
12:05",
                                                 "2026-11-15 23:50", "2026-11-16
00:10"])})
don["gio"] = don["thoi_gian"].dt.hour                          # giờ trong ngày
don["thu"] = don["thoi_gian"].dt.dayofweek                     # 0 = thứ Hai, 6 =
Chủ nhật
don["cuoi_tuan"] = (don["thu"] >= 5).astype(int)
don["gio_sin"] = np.sin(2 * np.pi * don["gio"] / 24).round(3) # mã hóa vòng tròn
(giải thích bên dưới)
don["gio_cos"] = np.cos(2 * np.pi * don["gio"] / 24).round(3)
print(don)
```

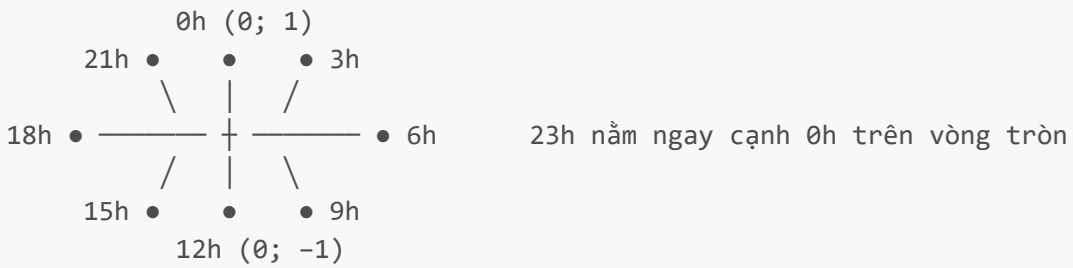
Kết quả mẫu:

```
            thoi_gian  gio  thu  cuoi_tuan  gio_sin  gio_cos
0 2026-11-09 07:30:00    7    0          0    0.966   -0.259
1 2026-11-14 12:05:00   12    5          1    0.000   -1.000
2 2026-11-15 23:50:00   23    6          1   -0.259    0.966
3 2026-11-16 00:10:00    0    0          0    0.000    1.000
```

Vì sao cần sin/cos?

Với cột `gio` thô: 23 giờ và 0 giờ cách nhau **23 đơn vị**, trong khi thực tế chỉ cách **20 phút**! 23 giờ và 12 giờ cách 11 đơn vị: "gần hơn" 0 giờ, vô lý.

Ý tưởng: đặt 24 giờ lên **mặt đồng hồ** (một vòng tròn). Mỗi giờ là một điểm có tọa độ (sin, cos). 23 giờ và 0 giờ nằm sát nhau trên mặt đồng hồ.




```
gio23 = don.loc[2, ["gio_sin", "gio_cos"]].values.astype(float)
gio0 = don.loc[3, ["gio_sin", "gio_cos"]].values.astype(float)
gio12 = don.loc[1, ["gio_sin", "gio_cos"]].values.astype(float)
print("Khoảng cách 23h–0h:", round(np.linalg.norm(gio23 - gio0), 3),
      "| 23h–12h:", round(np.linalg.norm(gio23 - gio12), 3))
```

Kết quả mẫu:

```
Khoảng cách 23h–0h: 0.261 | 23h–12h: 1.983
```

Giờ thì 23h và 0h **gần nhau** (0,261), 23h và 12h **xa nhau** (1,983). Đúng thực tế. Áp dụng tương tự cho: thứ trong tuần (chia 7), tháng trong năm (chia 12), hướng gió (chia 360°).

Các đặc trưng thời gian hay dùng khác: ngày lễ, ngày lương, mùa, "số ngày kể từ lần mua trước", "số đơn trong 7 ngày gần nhất".

---

6. Cột chữ có nhiều nhóm

Cột `purpose` (mục đích vay) của dữ liệu tín dụng có 10 nhóm, vài nhóm rất hiếm:

```
from sklearn.datasets import fetch_openml

credit = fetch_openml("credit-g", version=1, as_frame=True).frame
credit["vo_no"] = (credit["class"] == "bad").astype(int)
print(credit["purpose"].value_counts().to_dict())
```

Kết quả mẫu:

```
{'radio/tv': 280, 'new car': 234, 'furniture/equipment': 181, 'used car': 103,
'business': 97, 'education': 50, 'repairs': 22, 'domestic appliance': 12, 'other':
12, 'retraining': 9}
```

Với cột như "tỉnh/thành" (63 nhóm) hay "mã sản phẩm" (hàng nghìn nhóm), one-hot sẽ tạo ra quá nhiều cột thưa, và nhóm hiếm không đủ dữ liệu để học.

6.1. Gom nhóm hiếm

```
dem = credit["purpose"].value_counts()
hiem = dem[dem < 30].index                                    # nhóm dưới 30
khách là "hiếm"
credit["purpose_gon"] =
credit["purpose"].astype(str).where(~credit["purpose"].isin(hiem), "khac")
print(credit["purpose_gon"].value_counts().to_dict())
```

Kết quả mẫu:

```
{'radio/tv': 280, 'new car': 234, 'furniture/equipment': 181, 'used car': 103,
'business': 97, 'khac': 55, 'education': 50}
```

6.2. Frequency encoding (Mã hóa tần suất)

Thay mỗi nhóm bằng **tỉ lệ xuất hiện** của nó: `credit["purpose"].map(credit["purpose"].value_counts(normalize=True))`. Một cột duy nhất. Hữu ích khi "phổ biến hay hiếm" mang ý nghĩa.

6.3. Target encoding (Mã hóa theo đích)

Thay mỗi nhóm bằng **tỉ lệ vỡ nợ trung bình** của nhóm đó.

```
print(credit.groupby("purpose")["vo_no"].mean().round(3).sort_values().to_dict())
```

Kết quả mẫu:

```
{'retraining': 0.111, 'used car': 0.165, 'radio/tv': 0.221, 'furniture/equipment':
0.32, 'domestic appliance': 0.333, 'business': 0.351, 'repairs': 0.364, 'new car':
0.38, 'other': 0.417, 'education': 0.44}
```

Vay mua **xe cũ**: 16,5% vỡ nợ. Vay **học tập**: 44%. Một cột duy nhất chứa thông tin rủi ro của mục đích vay.

> ⚠️ **Rất dễ rò rỉ dữ liệu!** Tính tỉ lệ vỡ nợ trên **toàn bộ** dữ liệu rồi đưa vào mô hình = mô hình được "nhìn thấy đáp án" (bài 1.11). Nhóm hiếm còn tệ hơn: "retraining" chỉ có 9 khách, tỉ lệ 11% có thể chỉ là may rủi. Dùng `TargetEncoder` của Scikit-learn (từ bản 1.3): nó chỉ học trên train, tự **làm trơn** nhóm hiếm về phía trung bình chung, và dùng cross-fitting nội bộ khi `fit_transform` để giảm rò rỉ.

```
from sklearn.preprocessing import TargetEncoder
from sklearn.model_selection import train_test_split

y_c = credit["vo_no"]
Xtr, Xte, ytr, yte = train_test_split(credit[["purpose"]], y_c, test_size=0.2,
random_state=42, stratify=y_c)
te = TargetEncoder(random_state=42).fit(Xtr, ytr)        # chỉ học từ train
print(pd.DataFrame({"purpose": Xte["purpose"].values[:4], "ma_hoa":
te.transform(Xte)[:4, 0].round(3)}))
```

Kết quả mẫu:

```
    purpose  ma_hoa
0  business   0.329
1  used car   0.191
2  radio/tv   0.214
3  business   0.329
```

Giá trị mã hóa gần với tỉ lệ vỡ nợ thật của nhóm, nhưng đã được làm trơn (ví dụ "used car": 0,191 thay vì 0,165).

---

7. Đặc trưng nghiệp vụ

Đây là loại **mạnh nhất**, vì đến từ hiểu biết mà dữ liệu không tự nói ra. Với tín dụng, một nhân viên ngân hàng sẽ nghĩ ngay đến:

```
credit["tra_moi_thang"] = credit["credit_amount"] / credit["duration"]   # số tiền
phải trả mỗi tháng
credit["tre"] = (credit["age"] < 25).astype(int)                          # khách
trẻ, ít lịch sử tín dụng
print(credit.groupby(pd.qcut(credit["tra_moi_thang"], 4), observed=True)
["vo_no"].mean().round(3))
print(credit.groupby("tre")["vo_no"].mean().round(3))
```

Kết quả mẫu:

```
tra_moi_thang
(24.055, 89.6]         0.400
(89.6, 130.333]        0.276
(130.333, 206.183]     0.248
(206.183, 2482.667]    0.276
Name: vo_no, dtype: float64
tre
0    0.281
1    0.409
Name: vo_no, dtype: float64
```

- Khách **dưới 25 tuổi**: 40,9% vỡ nợ, so với 28,1%.
- Bất ngờ: nhóm trả **ít nhất** mỗi tháng lại rủi ro nhất (40%). Có thể vì trả ít mỗi tháng thường đi kèm **thời hạn rất dài** (bài 2.4: thời hạn dài là rủi ro), hoặc khoản vay nhỏ của người có thu nhập thấp. Một phát hiện cần trao đổi với chuyên gia nghiệp vụ.

Gợi ý đặc trưng nghiệp vụ cho các lĩnh vực khác:

Lĩnh vực	Đặc trưng
Thương mại điện tử	Số ngày từ lần mua cuối (Recency), số lần mua (Frequency), tổng chi (Monetary): bộ **RFM**
Tín dụng	Tỉ lệ nợ / thu nhập, số lần trễ hạn 12 tháng qua
Bất động sản	Giá trung bình các căn lân cận, khoảng cách tới trường học, chợ, metro
Giao hàng	Khoảng cách, giờ cao điểm, thời tiết

---

8. Kiểm chứng: đặc trưng mới có thật sự giúp?

Nguyên tắc: mỗi đặc trưng mới là một **giả thuyết**. Phải kiểm chứng bằng số liệu trên dữ liệu **chưa thấy**. Ở đây ta dùng **cross-validation** 5 phần: chia train thành 5 phần, lần lượt lấy 1 phần làm kiểm tra, rồi lấy trung bình (sẽ học kỹ ở bài 2.17).

8.1. Giá nhà

```
from sklearn.model_selection import KFold, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
import lightgbm as lgb

goc = ["kinh_do", "vi_do", "tuoi_nha", "tong_phong", "tong_phong_ngu", "dan_so",
"so_ho", "thu_nhap"]
day_du = goc + ["phong_moi_ho", "ti_le_phong_ngu", "nguoi_moi_ho", "km_LA",
"km_SF", "km_tp_lon",
                "log_phong_moi_ho", "log_nguoi_moi_ho", "log_km_tp_lon",
"log_thu_nhap"]
gon = goc + ["ti_le_phong_ngu", "log_phong_moi_ho", "log_nguoi_moi_ho",
"log_km_tp_lon", "log_thu_nhap"]

cv = KFold(n_splits=5, shuffle=True, random_state=42)       # shuffle=True rất
quan trọng (bài 2.17)
for ten, cot in [("8 cột gốc", goc), ("18 cột (tỉ lệ thô + log)", day_du), ("13
cột (chỉ dùng log)", gon)]:
    lr = cross_val_score(make_pipeline(StandardScaler(), LinearRegression()),
df[cot], y, cv=cv, scoring="r2")
    gb = cross_val_score(lgb.LGBMRegressor(n_estimators=500, learning_rate=0.05,
random_state=42, verbose=-1),
                         df[cot], y, cv=cv, scoring="r2")
    print(f"{ten:<26}: R² LinReg = {lr.mean():.3f} (phần kém nhất {lr.min():.3f})
| R² LightGBM = {gb.mean():.3f}")
```

Kết quả mẫu (khoảng 1 phút):

```
8 cột gốc                : R² LinReg = 0.602 (phần kém nhất 0.577) | R² LightGBM
= 0.817
18 cột (tỉ lệ thô + log)  : R² LinReg = 0.576 (phần kém nhất 0.256) | R² LightGBM
= 0.829
13 cột (chỉ dùng log)     : R² LinReg = 0.653 (phần kém nhất 0.642) | R² LightGBM
= 0.827
```

Ba bài học từ một bảng:

1. **Đặc trưng tốt giúp cả hai loại mô hình**: bộ 13 cột tăng R^2 Linear Regression từ 0,602 lên **0,653**, LightGBM từ 0,817 lên **0,827**.

2. **Đặc trưng "đúng ý tưởng" nhưng "sai hình dạng" có thể gây hại cho mô hình tuyến tính.** Bộ 18 cột có các tỉ lệ **thô** với outlier cực đoan (141 phòng/hộ, 502 người/hộ). Trong một phần dữ liệu, vài khu cực đoan đó làm R^2 sụp xuống **0,256**. Mô hình cây không bị ảnh hưởng (chỉ hỏi "lớn hơn ngưỡng?"). Thay bằng **log**, Linear Regression ổn định trở lại.
3. **Luôn nhìn cả phần kém nhất**, không chỉ trung bình: trung bình 0,576 che giấu một lần sụp đổ.

8.2. Tín dụng

```
from sklearn.model_selection import StratifiedKFold
from sklearn.linear_model import LogisticRegression

co_ban = pd.get_dummies(credit.drop(columns=["class", "vo_no", "purpose_gon",
"tra_moi_thang", "tre"]),
                        drop_first=True, dtype=int)
them = co_ban.assign(tra_moi_thang=credit["tra_moi_thang"],
                     log_so_tien=np.log(credit["credit_amount"]),
                     tre=credit["tre"])
cv_c = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
for ten, X in [("48 cột cơ bản", co_ban), ("+3 đặc trưng mới", them)]:
    s = cross_val_score(make_pipeline(StandardScaler(),
LogisticRegression(max_iter=2000)), X, y_c, cv=cv_c, scoring="roc_auc")
    print(f"{ten:<18}: AUC = {s.mean():.3f} ± {s.std():.3f}")
```

Kết quả mẫu:

```
48 cột cơ bản     : AUC = 0.781 ± 0.021
+3 đặc trưng mới  : AUC = 0.782 ± 0.021
```

Không cải thiện (0,781 → 0,782, nhỏ hơn rất nhiều so với độ dao động ±0,021). Vì sao? Thông tin "trả mỗi tháng" đã nằm trong `credit_amount` và `duration`; "trẻ" đã nằm trong `age`. Với 1.000 khách, mô hình đã tự khai thác được.

Đây là kết quả bình thường. Phần lớn ý tưởng đặc trưng mới sẽ **không** giúp. Người làm ML giỏi thử nhiều, kiểm chứng nghiêm túc, và **chỉ giữ lại** những gì thật sự có ích. Đặc trưng thừa làm mô hình chậm hơn, khó giải thích hơn, và dễ overfit hơn.

---

9. Checklist Feature Engineering

```
□ Tỉ lệ: chia các cột "tổng" cho cột "quy mô" (theo đầu người, theo hộ, theo
m²...)
□ Log: cho cột lệch phải mạnh (giá, thu nhập, dân số, số lượt xem)
□ Chia nhóm: khi quan hệ có dạng bậc thang/chữ U (chủ yếu cho mô hình tuyến tính)
□ Ngày giờ: tách giờ, thứ, tháng, cuối tuần, ngày lễ; mã hóa vòng tròn sin/cos
□ Cột chữ nhiều nhóm: gom nhóm hiếm; frequency / target encoding (cẩn thận rò rỉ!)
□ Tương tác: tích hoặc tỉ lệ giữa các cột có ý nghĩa nghiệp vụ
□ Địa lý: khoảng cách tới điểm quan trọng; cụm (DBSCAN, K-Means)
```

```
□ Nghiệp vụ: hỏi chuyên gia "Bạn nhìn vào gì để quyết định?"
□ Kiểm chứng: cross-validation, so với bộ cơ bản, nhìn cả phần kém nhất
□ Viết thành HÀM để áp dụng y hệt cho dữ liệu mới (đưa vào Pipeline ở bài 2.17)
```

---

10. Bài tập

1. Tạo đặc trưng "góc nhìn" cho giá nhà: `kinh_do + vi_do` và `kinh_do - vi_do` (hai đường chéo của bản đồ). Thêm vào bộ 13 cột. R^2 Linear Regression thay đổi thế nào?
2. Với dữ liệu tín dụng, tạo `muc_dich_rui_ro` bằng `TargetEncoder` cho cột `purpose` (đặt **bên trong** cross-validation bằng pipeline, gợi ý: `ColumnTransformer`, sẽ học ở bài 2.17). AUC có tăng không?
3. Mã hóa vòng tròn cho **thứ trong tuần** (0–6). Tính khoảng cách giữa Chủ nhật (6) và thứ Hai (0), so với Chủ nhật và thứ Năm (3).
4. Suy nghĩ: bạn có dữ liệu đơn hàng Shopee (mã khách, thời điểm, số tiền, danh mục). Đề xuất 5 đặc trưng để dự đoán **"khách có mua lại trong 30 ngày tới không"**.

Đáp án

> Mẹo Notion: chọn tiêu đề "Đáp án" → `Turn into → Toggle heading 3`.

1. Code:

```
df["cheo_1"] = df["kinh_do"] + df["vi_do"]
df["cheo_2"] = df["kinh_do"] - df["vi_do"]
s = cross_val_score(make_pipeline(StandardScaler(), LinearRegression()), df[gon +
["cheo_1", "cheo_2"]], y, cv=cv, scoring="r2")
print(round(s.mean(), 3))
```

R^2 **không đổi** (0,653). Ý tưởng không tệ: bờ biển California chạy chéo từ tây bắc xuống đông nam, nên hướng chéo có ý nghĩa. Nhưng mô hình tuyến tính đã có sẵn `kinh_do` và `vi_do`, nó **tự cộng trừ được** hai cột đó với hệ số bất kỳ, nên tổ hợp tuyến tính của chúng không mang thêm thông tin gì. Bài học: với mô hình tuyến tính, đặc trưng mới phải là **phi tuyến** so với các cột đã có (tích, tỉ lệ, log, khoảng cách...) mới giúp được. 2. Gợi ý pipeline:

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
cot_chu_khac = [c for c in credit.select_dtypes("category").columns if c not in
["class", "purpose"]]
cot_so = credit.drop(columns=["class",
"vo_no"]).select_dtypes("number").columns.tolist()
tien_xu_ly = ColumnTransformer([
    ("so", StandardScaler(), cot_so),
    ("chu", OneHotEncoder(handle_unknown="ignore"), cot_chu_khac),
    ("muc_dich", TargetEncoder(random_state=42), ["purpose"]),
])
mh = make_pipeline(tien_xu_ly, LogisticRegression(max_iter=2000))
s = cross_val_score(mh, credit.drop(columns=["class", "vo_no"]), y_c, cv=cv_c,
```

```
    scoring="roc_auc")
print(round(s.mean(), 3))
```

Ghi lại kết quả. Với 10 nhóm, one-hot và target encoding thường cho kết quả gần nhau; target encoding tỏa sáng khi cột có hàng trăm, hàng nghìn nhóm. 3. Chủ nhật: (sin(2π·6/7), cos(2π·6/7)) ≈ (−0,782; 0,623). Thứ Hai: (0; 1). Thứ Năm: (sin(2π·3/7), cos(2π·3/7)) ≈ (0,434; −0,901).

- CN – T2: $\sqrt{(0{,}782^2 + 0{,}377^2)} \approx$ **0,868**
- CN – T5: $\sqrt{(1{,}216^2 + 1{,}524^2)} \approx$ **1,950**

Chủ nhật gần thứ Hai hơn thứ Năm, đúng với vòng tuần. 4. Gợi ý: (1) số ngày từ lần mua gần nhất (Recency); (2) số đơn trong 90 ngày (Frequency); (3) tổng chi 90 ngày (Monetary); (4) khoảng cách trung bình giữa các lần mua; (5) tỉ lệ đơn mua trong đợt sale (khách "săn sale" hay không); thêm: danh mục mua nhiều nhất, đơn gần nhất có bị hoàn/hủy không. Quan trọng: mọi đặc trưng chỉ được tính từ dữ liệu **trước** thời điểm dự đoán, nếu không sẽ rò rỉ tương lai.

---

11. Flashcard

Mặt trước	Mặt sau
Feature Engineering là gì?	Dùng hiểu biết nghiệp vụ để tạo/biến đổi/chọn đặc trưng giúp mô hình học dễ hơn.
Vì sao dùng tỉ lệ thay cột tổng?	Cột tổng phụ thuộc quy mô; tỉ lệ phản ánh đặc điểm thật.
Log giúp gì?	Nén giá trị cực đoan, làm lộ quan hệ bị outlier che.
Vì sao mã hóa giờ bằng sin/cos?	Để 23h và 0h nằm gần nhau như trên mặt đồng hồ.
Target encoding là gì?	Thay nhóm bằng giá trị trung bình của cột đích trong nhóm.
Nguy hiểm của target encoding?	Rò rỉ dữ liệu; nhóm hiếm cho giá trị không đáng tin.
Mô hình nào bị hại bởi tỉ lệ có outlier cực đoan?	Mô hình tuyến tính. Mô hình cây ít bị ảnh hưởng.
Đặc trưng mới không giúp thì sao?	Bình thường; bỏ đi. Chỉ giữ những gì kiểm chứng có ích.
Bộ RFM là gì?	Recency, Frequency, Monetary: 3 đặc trưng kinh điển về hành vi mua hàng.
Vì sao viết đặc trưng thành hàm?	Để áp dụng y hệt cho dữ liệu mới/test, tránh sai lệch.

Bài tiếp theo: 2.17 – Pipeline & Cross-validation.